

Metodologie di Programmazione (A-L)

II semestre - a.a. 2025 – 2026

Ereditarietà I

a cura di Massimo La Morgia



SAPIENZA
UNIVERSITÀ DI ROMA

*Tutti i diritti relativi al presente materiale didattico ed al suo contenuto sono riservati a Sapienza e ai suoi autori (o docenti che lo hanno prodotto). È consentito l'uso personale dello stesso da parte dello studente a fini di studio. Ne è vietata nel modo più assoluto la diffusione, duplicazione, cessione, trasmissione, distribuzione a terzi o al pubblico pena le sanzioni applicabili per legge.

**I crediti sulle slide di questo corso sono riportati nell'ultima slide

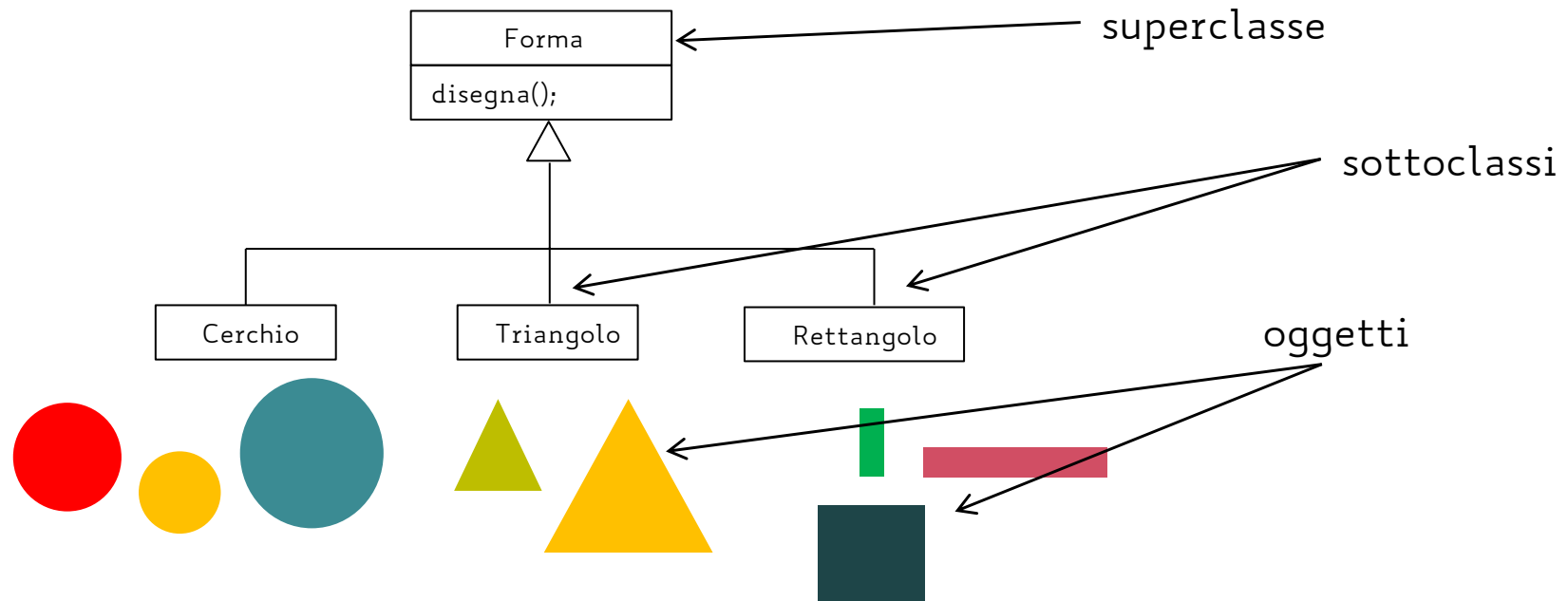
Ereditarietà parte I

Ereditarietà

- Un **concetto cardine** della programmazione orientata agli oggetti
- Una **forma di riuso del software** in cui una classe è creata:
 - “assorbendo” i **membri di una classe esistente**
 - aggiungendo **nuove caratteristiche** o migliorando quelle esistenti
- **Programmazione mattone su mattone**
 - Detto anche: «**non si butta via niente**»
- Aumenta le probabilità che il sistema sia **implementato e mantenuto in maniera efficiente**

Molti tipi di "forma"

- Si può progettare una classe **Forma** che rappresenta una forma generica e poi **specializzarla estendendo** la classe



Molti tipi di "forma"

Estende la classe Forma

```
public class Forma
{
    public void disegna() { }
}
```

```
public class Triangolo extends Forma
{
    private double base;
    private double altezza;

    public Triangolo(double base, double altezza)
    {
        this.base = base;
        this.altezza = altezza;
    }

    public double getBase()
    {
        return base;
    }

    public double getAltezza()
    {
        return altezza;
    }
}
```

```
public class Cerchio extends Forma
{
    /**
     * Raggio del cerchio
     */
    private double raggio;

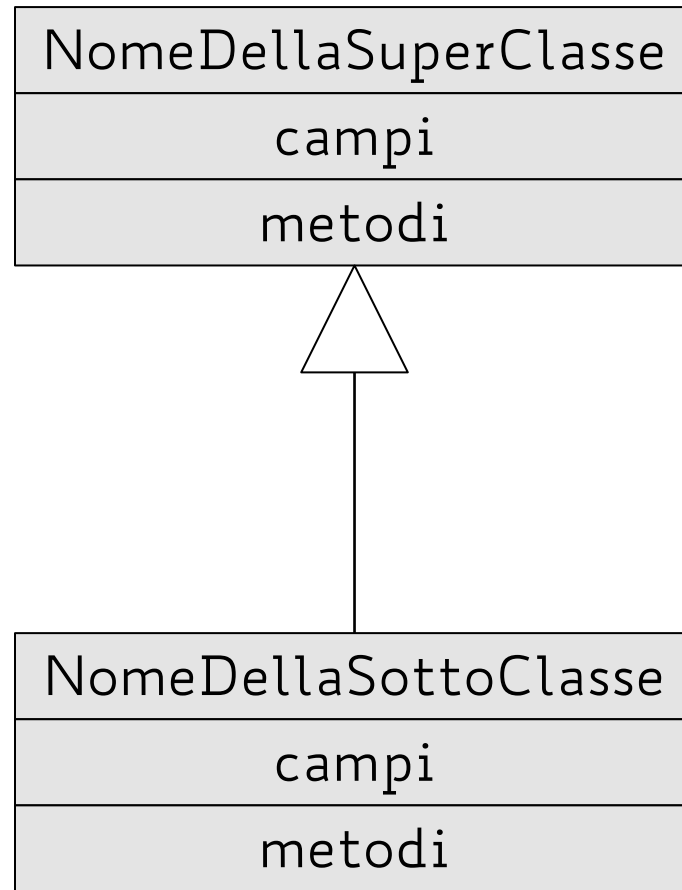
    public Cerchio(int raggio)
    {
        this.raggio = raggio;
    }

    public double getRaggio() { return raggio; }
    public double getCirconferenza() { return 2*Math.PI*raggio; }
}
```

Ereditarietà: che cosa si eredita?

- Una **sottoclasse** estende la **superclasse**
- La **sottoclasse** eredita i **membri** della **superclasse**
 - **campi** e **metodi d'istanza** secondo il livello di accesso specificato (**riuso del codice**)
- Inoltre la **sottoclasse** può:
 - aggiungere **nuovi metodi** e **campi**
 - **ridefinire (sovrascrittura)** i metodi che eredita dalla superclasse
(ma **NON** può ridefinire i campi)

La relazione di estensione in UML



Un esempio: Bubble Bobble

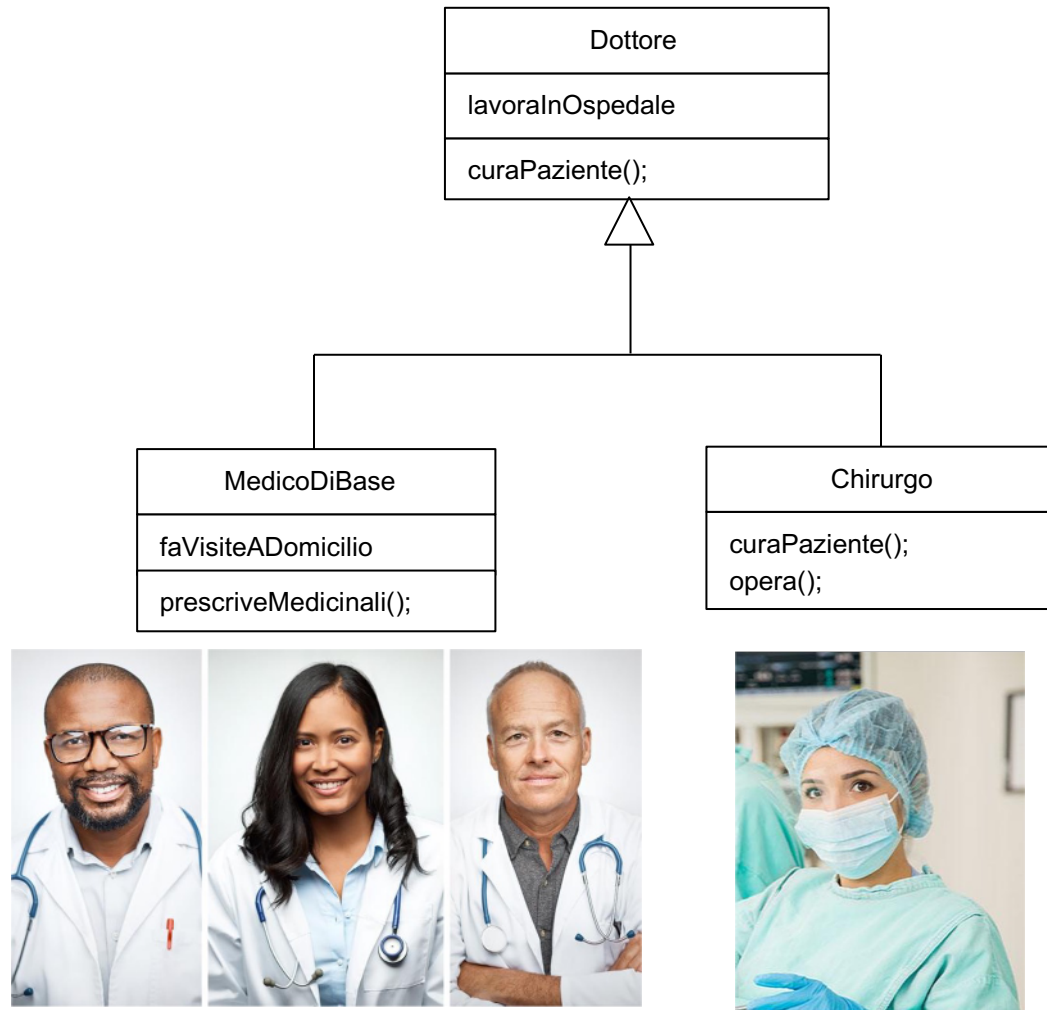
- Abbiamo tanti "oggetti" (tante entità) in senso lato:
 - Piattaforme
 - Bolle
 - Bonus
- Alcuni sono "personaggi":
 - I giocatori (draghetti)
 - I nemici



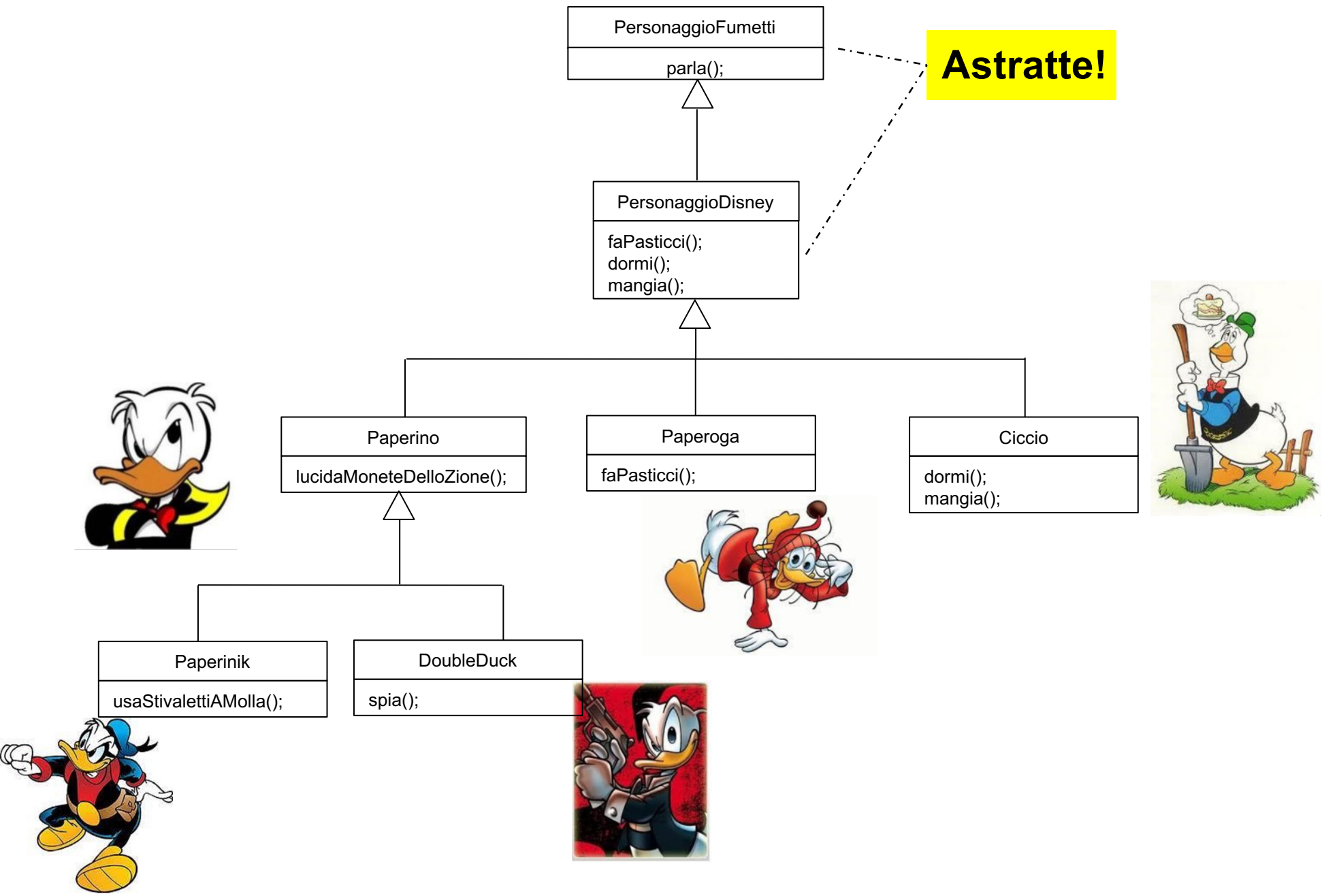
Diventerà «naturale» porsi le seguenti domande:

- Che cosa hanno in comune tutti?
- E che cosa li distingue?

Esempio: il dottore, il medico di base e il chirurgo



Esempio: paperino & company



Classi astratte

- Una classe **astratta** (definita mediante la parola chiave **abstract**) non può essere istanziata
 - Quindi **NON** possono esistere **oggetti** per quella classe

```
/**
 * Classe astratta: non e' possibile istanziarla
 */
public abstract class PersonaggioDisney
{
    /**
     * Metodo astratto senza implementazione
     */
    abstract void faPasticci();
}
```

Classi astratte

- Una classe **astratta** (definita mediante la parola chiave **abstract**) non può essere istanziata
 - Quindi **NON** possono esistere **oggetti** per quella classe

```
/**
 * Classe astratta: non e' possibile istanziarla
 */
public abstract class PersonaggioDisney
{
    /**
     * Metodo astratto senza implementazione
     */
    abstract void faPasticci();
}
```

- Tipicamente verrà **estesa** da **altre classi**, che invece potranno essere istanziate

```
public class Paperoga extends PersonaggioDisney
{
    public void faPasticci()
    {
        System.out.println("bla bla bla bla bla bla bla");
    }
}
```

Metodi astratti

- Anche i metodi possono essere definiti **astratti**
 - Esclusivamente all'interno di una **classe dichiarata astratta**
- **NON** forniscono l'implementazione per quel metodo

```
/**
 * Classe astratta: non e' possibile istanziarla
 */
public abstract class PersonaggioDisney
{
    /**
     * Metodo astratto senza implementazione
     */
    abstract void faPasticci();
}
```

Metodi astratti

- Anche i metodi possono essere definiti **astratti**
 - Esclusivamente all'interno di una **classe** dichiarata **astratta**
- **NON** forniscono l'implementazione per quel metodo

```
/**
 * Classe astratta: non e' possibile istanziarla
 */
public abstract class PersonaggioDisney
{
    /**
     * Metodo astratto senza implementazione
     */
    abstract void faPasticci();
}
```

- Impongono alle sottoclassi **non astratte** di implementare il metodo

```
public class Paperoga extends PersonaggioDisney
{
    public void faPasticci()
    {
        System.out.println("bla bla bla bla bla bla bla");
    }
}
```

this e super

- La parola chiave **this** usata come nome di metodo **obbligatoriamente** nella **prima riga del costruttore** permette di richiamare un altro **costruttore della stessa classe**
- La parola chiave **super** usata come nome di metodo **obbligatoriamente** nella prima riga del costruttore permette di richiamare un **costruttore della superclasse**
- Ogni sottoclasse deve **esplicitamente definire un costruttore** se la superclasse **NON fornisce un costruttore senza argomenti**
 - Questo avviene se si definisce un costruttore **con almeno un argomento** ma non si definisce il costruttore senza argomenti
- **Nota bene:** anche le classi astratte possono avere costruttori! **Perché?**

Esempio di dinamica delle chiamate a costruttori

- Definiamo una piccola gerarchia di tre classi X, Y, Z:

```
public class X
{
    public X(int k)
    {
        System.out.println("X(int k)");
    }

    public X()
    {
        System.out.println("X()");
    }
}

public class Y extends X
{
    public Y(int k)
    {
        System.out.println("Y(int k)");
    }
}
```

```
public class Z extends Y
{
    public Z(int k)
    {
        super(k);
        System.out.println("Z(int k)");
    }

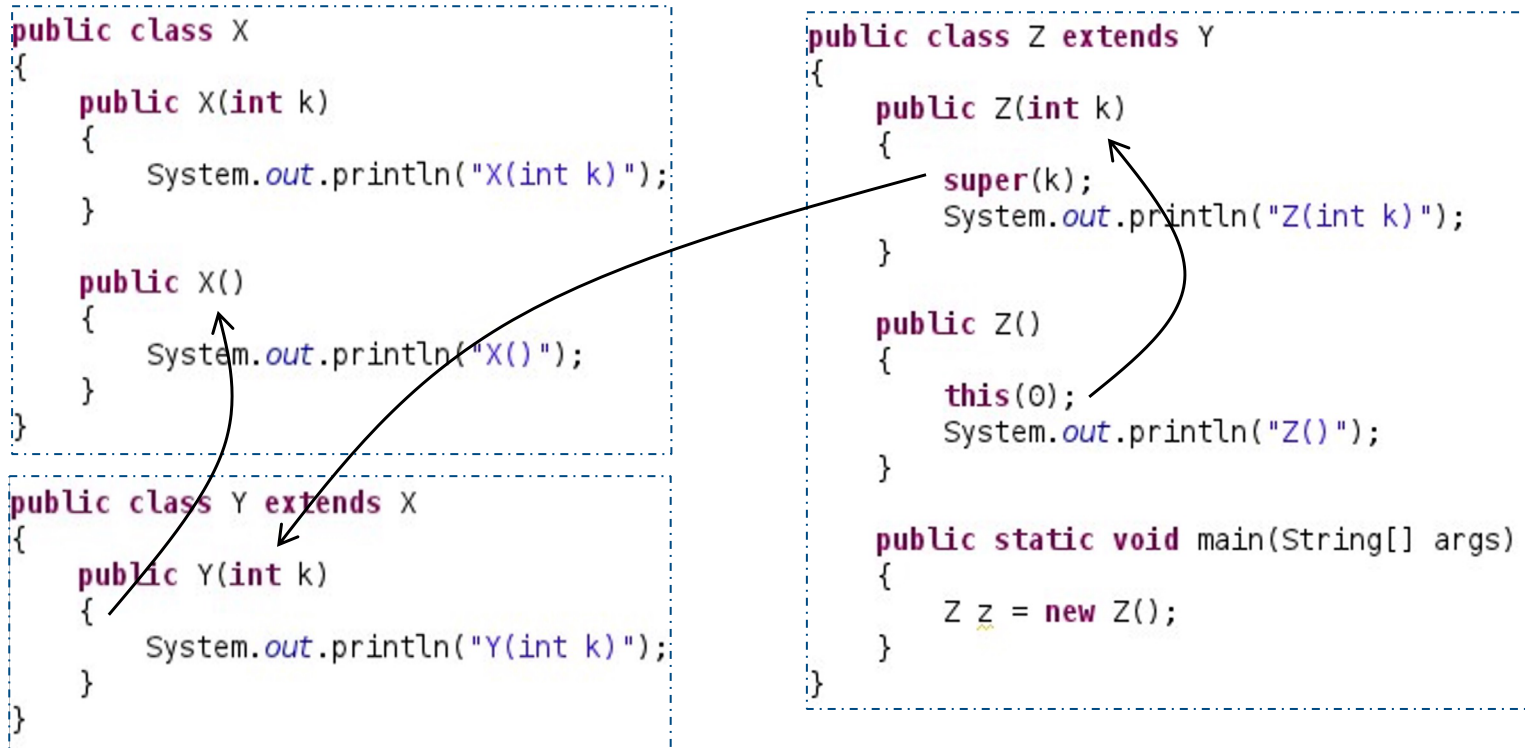
    public Z()
    {
        this(0);
        System.out.println("Z()");
    }

    public static void main(String[] args)
    {
        Z z = new Z();
    }
}
```

Esercizio: Disegnate il diagramma delle classi in UML

Esempio di dinamica delle chiamate a costruttori

- Definiamo una piccola gerarchia di tre classi X, Y, Z:



```
public class X
{
    public X(int k)
    {
        System.out.println("X(int k)");
    }

    public X()
    {
        System.out.println("X()");
    }
}

public class Y extends X
{
    public Y(int k)
    {
        System.out.println("Y(int k)");
    }
}

public class Z extends Y
{
    public Z(int k)
    {
        super(k);
        System.out.println("Z(int k)");
    }

    public Z()
    {
        this(0);
        System.out.println("Z()");
    }

    public static void main(String[] args)
    {
        Z z = new Z();
    }
}
```

Nell'eseguire l'istruzione `new Z();` abbiamo in output:

```
X()
Y(int k)
Z(int k)
Z()
```

Esempio di dinamica delle chiamate a costruttori

```
public class X  
{  
}
```

```
public class Y extends X  
{  
}
```

Costruttori di default in X e Y



```
public class X  
{  
    public X()  
    {  
    }  
}
```

```
public class Y extends X  
{  
}
```

Costruttore esplicito con zero argomenti per X e di default per Y (quello di Y chiama automaticamente quello di X)



```
public class X  
{  
}
```

```
public class Y extends X  
{  
    public Y(int k)  
    {  
    }  
}
```

Costruttore di default in X e con un argomento in Y (chiama in automatico il costruttore di default di X)



Esempio di dinamica delle chiamate a costruttori

```
public class X
{
    public X(int k)
    {
    }
}

public class Y extends X
{
}
```

Costruttore con un parametro per X e di default per Y



```
public class X
{
    public X()
    {
    }

    public X(int k)
    {
    }
}

public class Y extends X
{
}
```

Costruttori con zero e un parametro per X e di default per Y (quello di Y chiama automaticamente quello di X)



Credits

Le slide di questo corso sono basate sulle slide del Prof. Faralli che sono a loro volta una rielaborazione delle slide del Prof. Navigli

In aggiunta, le slide sono state revisionate dagli studenti borsisti della Facoltà di Ingegneria Informatica, Informatica e Statistica: Mario Marra e Paolo Straniero.

Font ad alta leggibilità Biancoenero® di biancoenero edizioni srl, disegnata da Umberto Mischì, con la consulenza di Alessandra Finzi, Daniele Zanoni e Luciano Perondi (Brevetto n. RM20110000128).

Disponibile gratuitamente per tutte le istituzioni e i privati che la utilizzino per scopi non commerciali.